

Module System in Node.js

Introduction

Modules are the building blocks of any Node.js application. They allow you to organize code into separate files, making your codebase more maintainable, reusable, and easier to understand. Think of modules like chapters in a book—each chapter (module) covers a specific topic and can be referenced from other chapters.

Node.js supports two module systems:

- **CommonJS (CJS)**: The original Node.js module system, using `require()` and `module.exports`
- **ES Modules (ESM)**: The modern JavaScript standard, using `import` and `export`

Understanding both systems is crucial because you'll encounter both in real-world projects, and knowing when to use each will make you a more effective Node.js developer.

1. CommonJS: The Traditional Node.js Module System

What is CommonJS?

CommonJS (CJS) is the original module system built into Node.js. It was created before JavaScript had a standard module system. CommonJS modules load synchronously and are evaluated at runtime, making them simple and straightforward to use.

Basic Module Creation and Usage

Example 1: Creating Your First Module

Let's create a simple utility module that performs mathematical operations.

```
// math.js - A simple module
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

function multiply(a, b) {
  return a * b;
}

// Export functions so other files can use them
module.exports = {
  add,
  subtract,
  multiply
};
```

```
// app.js - Using the module
const math = require('./math.js');

console.log('Addition: 5 + 3 =', math.add(5, 3));
console.log('Subtraction: 10 - 4 =', math.subtract(10, 4));
console.log('Multiplication: 6 * 7 =', math.multiply(6, 7));
```

Output:

```
Addition: 5 + 3 = 8
Subtraction: 10 - 4 = 6
Multiplication: 6 * 7 = 42
```

Key Points:

- `module.exports` is an object that contains what you want to share from the module
- `require()` loads the module and returns whatever was assigned to `module.exports`

- Use `./` for local modules (files in your project)
- Module code executes once when first required, then cached

Example 2: Different Export Patterns

There are multiple ways to export values from a module. Let's explore them progressively.

```
// Pattern 1: Exporting an object (most common)
// user-service.js
const users = [];

function createUser(name, email) {
  const user = { id: users.length + 1, name, email };
  users.push(user);
  return user;
}

function getAllUsers() {
  return users;
}

function getUserById(id) {
  return users.find(user => user.id === id);
}

module.exports = {
  createUser,
  getAllUsers,
  getUserById
};
```

```
// Pattern 2: Exporting a single function
// logger.js
function log(message) {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${message}`);
}

module.exports = log;
```

```
// Pattern 3: Exporting a class
// Product.js
class Product {
  constructor(name, price) {
    this.name = name;
    this.price = price;
    this.id = Math.random().toString(36).substr(2, 9);
  }

  getDetails() {
    return `${this.name} - ${this.price}`;
  }

  applyDiscount(percentage) {
    this.price = this.price * (1 - percentage / 100);
    return this.price;
  }
}

module.exports = Product;
```

```
// Using all three patterns
// main.js
const userService = require('./user-service');
const log = require('./logger');
const Product = require('./Product');

// Using the user service
log('Creating users...');
userService.createUser('Alice', 'alice@example.com');
userService.createUser('Bob', 'bob@example.com');

const users = userService.getAllUsers();
console.log('All users:', users);

// Using the logger
log('Users created successfully');

// Using the Product class
const product = new Product('Laptop', 999);
console.log('Product:', product.getDetails());
```

```
product.applyDiscount(10);  
console.log('After discount:', product.getDetails());
```

Output:

```
[2026-02-17T10:30:00.000Z] Creating users...  
All users: [  
  { id: 1, name: 'Alice', email: 'alice@example.com' },  
  { id: 2, name: 'Bob', email: 'bob@example.com' }  
]  
[2026-02-17T10:30:00.000Z] Users created successfully  
Product: Laptop - $999  
After discount: Laptop - $899.1
```

Example 3: Using exports Shorthand

You can also use `exports` as a shorthand, but be careful—it has limitations.

```
// calculator.js - Using exports shorthand  
exports.square = function(n) {  
  return n * n;  
};  
  
exports.cube = function(n) {  
  return n * n * n;  
};  
  
exports.PI = 3.14159;  
  
// ✗ This WON'T work - reassigning exports breaks the reference  
// exports = { square, cube, PI }; // Don't do this!  
  
// ✓ This works - adding properties to exports  
// exports.newFunction = ...
```

```
// using-calculator.js
const calculator = require('./calculator');

console.log('Square of 5:', calculator.square(5));
console.log('Cube of 3:', calculator.cube(3));
console.log('Value of PI:', calculator.PI);
```

Output:

```
Square of 5: 25
Cube of 3: 27
Value of PI: 3.14159
```

Important Rule: You can add properties to `exports`, but you cannot reassign it. If you need to export a single value or completely replace the exports object, use `module.exports`.

Example 4: Module Caching

Modules are cached after the first time they are loaded. This is an important performance optimization to understand.

```
// counter.js - Demonstrates caching
let count = 0;

function increment() {
  count++;
  return count;
}

function getCount() {
  return count;
}

console.log('Counter module loaded!');

module.exports = { increment, getCount };
```

```
// test-caching.js
console.log('First require:');
const counter1 = require('./counter');

console.log('\nSecond require:');
const counter2 = require('./counter');

console.log('\nAre they the same?', counter1 === counter2);

console.log('\nIncrementing with counter1:');
console.log('Count:', counter1.increment());
console.log('Count:', counter1.increment());

console.log('\nChecking count from counter2:');
console.log('Count:', counter2.getCount());
```

Output:

```
First require:
Counter module loaded!

Second require:

Are they the same? true

Incrementing with counter1:
Count: 1
Count: 2

Checking count from counter2:
Count: 2
```

Key Insight: The module code only runs once (notice "Counter module loaded!" appears only once). Both `counter1` and `counter2` reference the same module instance, so they share state.

Example 5: Requiring Built-in and NPM Modules

```
// working-with-modules.js

// Built-in modules (no ./ needed, no installation required)
const fs = require('fs');
const path = require('path');
const os = require('os');

// NPM modules (installed via npm install)
// const express = require('express');
// const lodash = require('lodash');

// Local modules (use ./ or ../)
const userService = require('./user-service');

// Using built-in modules
console.log('Operating System:', os.platform());
console.log('Home Directory:', os.homedir());
console.log('CPU Cores:', os.cpus().length);

const filePath = path.join(__dirname, 'data', 'users.txt');
console.log('File path:', filePath);

// Check if file exists (synchronous for demo)
if (fs.existsSync('./math.js')) {
  console.log('math.js exists in current directory');
}
```

Output:

```
Operating System: linux
Home Directory: /home/claude
CPU Cores: 4
File path: /home/claude/data/users.txt
math.js exists in current directory
```

Module Resolution Rules:

- No prefix (e.g., 'fs') → Built-in or NPM module

- `./` prefix (e.g., `'./math'`) → Local module in same directory
- `../` prefix (e.g., `'../utils/helper'`) → Local module in parent directory
- File extension is optional for `.js` files

2. ES Modules: The Modern JavaScript Standard

What are ES Modules?

ES Modules (ESM) are the official JavaScript module system, standardized in ES6 (ECMAScript 2015). They use `import` and `export` statements and are the future of JavaScript modules. Node.js added full support for ES modules starting from version 12.

Enabling ES Modules in Node.js

To use ES modules in Node.js, you have two options:

1. Add `"type": "module"` to your `package.json`
2. Use the `.mjs` file extension

Example 6: Setting Up ES Modules

```
// package.json
{
  "name": "my-project",
  "version": "1.0.0",
  "type": "module",
  "description": "Project using ES Modules"
}
```

Note: Once you set `"type": "module"`, all `.js` files in your project are treated as ES modules. You must use `import / export` instead of `require / module.exports`.

Example 7: Basic ES Module Export and Import

```
// utils.js - Named exports
export function formatDate(date) {
  return date.toISOString().split('T')[0];
}

export function capitalize(str) {
  return str.charAt(0).toUpperCase() + str.slice(1);
}

export const APP_NAME = 'MyApp';
export const VERSION = '1.0.0';
```

```
// app.js - Importing named exports
import { formatDate, capitalize, APP_NAME, VERSION } from './utils.js';

console.log('App Name:', APP_NAME);
console.log('Version:', VERSION);

const today = new Date();
console.log('Formatted Date:', formatDate(today));
console.log('Capitalized:', capitalize('hello world'));
```

Output:

```
App Name: MyApp
Version: 1.0.0
Formatted Date: 2026-02-17
Capitalized: Hello world
```

Important Differences from CommonJS:

- Must include file extension in imports (`./utils.js` not `./utils`)
- Imports are read-only—you cannot reassign imported values
- Imports are hoisted—they execute before other code
- ES modules are always in strict mode

Example 8: Default Exports

```
// Database.js - Default export
class Database {
  constructor(connectionString) {
    this.connectionString = connectionString;
    this.connected = false;
  }

  async connect() {
    console.log(`Connecting to ${this.connectionString}...`);
    // Simulate connection delay
    await new Promise(resolve => setTimeout(resolve, 1000));
    this.connected = true;
    console.log('Connected!');
  }

  async query(sql) {
    if (!this.connected) {
      throw new Error('Database not connected');
    }
    console.log(`Executing: ${sql}`);
    return { rows: [], count: 0 };
  }
}

// Default export - only one per file
export default Database;
```

```
// Using default export
import Database from './Database.js';

const db = new Database('mongodb://localhost:27017/myapp');

async function run() {
  await db.connect();
  const result = await db.query('SELECT * FROM users');
  console.log('Query result:', result);
}

run();
```

Output:

```
Connecting to mongodb://localhost:27017/myapp...
Connected!
Executing: SELECT * FROM users
Query result: { rows: [], count: 0 }
```

Example 9: Mixing Named and Default Exports

```
// api-client.js - Both named and default exports
const API_BASE_URL = 'https://api.example.com';

export class ApiError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.name = 'ApiError';
  }
}

export function buildUrl(endpoint) {
  return `${API_BASE_URL}${endpoint}`;
}

class ApiClient {
  constructor(apiKey) {
    this.apiKey = apiKey;
  }

  async get(endpoint) {
    const url = buildUrl(endpoint);
    console.log(`GET ${url}`);
    return { data: 'Sample response' };
  }

  async post(endpoint, data) {
    const url = buildUrl(endpoint);
    console.log(`POST ${url}`, data);
    return { success: true };
  }
}
```

```
// Default export
export default ApiClient;
```

```
// Using mixed exports
import ApiClient, { ApiError, buildUrl } from './api-client.js';

const client = new ApiClient('my-secret-key');

async function testApi() {
  try {
    const url = buildUrl('/users');
    console.log('Built URL:', url);

    const response = await client.get('/users/1');
    console.log('Response:', response);

    await client.post('/users', { name: 'Alice' });
  } catch (error) {
    if (error instanceof ApiError) {
      console.error('API Error:', error.statusCode, error.message);
    }
  }
}

testApi();
```

Output:

```
Built URL: https://api.example.com/users
GET https://api.example.com/users/1
Response: { data: 'Sample response' }
POST https://api.example.com/users { name: 'Alice' }
```

Example 10: Import Aliases and Namespace Imports

```
// math-operations.js
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

export function multiply(a, b) {
  return a * b;
}

export function divide(a, b) {
  if (b === 0) throw new Error('Division by zero');
  return a / b;
}
```

```
// Method 1: Import specific functions with aliases
import { add as sum, multiply as times } from './math-operations.js';

console.log('Method 1: Aliases');
console.log('5 + 3 =', sum(5, 3));
console.log('4 * 6 =', times(4, 6));

// Method 2: Import all as namespace
import * as Math from './math-operations.js';

console.log('\nMethod 2: Namespace');
console.log('10 - 3 =', Math.subtract(10, 3));
console.log('15 / 3 =', Math.divide(15, 3));
console.log('Available functions:', Object.keys(Math));
```

Output:

Method 1: Aliases

5 + 3 = 8

4 * 6 = 24

Method 2: Namespace

10 - 3 = 7

15 / 3 = 5

Available functions: ['add', 'subtract', 'multiply', 'divide']

When to Use Each:

- **Specific imports:** When you only need a few functions
- **Aliases:** When names conflict or you want clearer names
- **Namespace imports:** When you need many functions and want them grouped

3. CommonJS vs ES Modules: Side-by-Side Comparison

Key Differences

Feature	CommonJS (CJS)	ES Modules (ESM)
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Loading	Synchronous, runtime	Asynchronous, compile-time
File Extension	.js (default in Node.js)	.js (with "type": "module") or .mjs
Import Location	Anywhere in code	Top-level only (except dynamic imports)
Conditional Loading	☒ Easy with if statements	☒ Not directly (use dynamic import)
Tree Shaking	☒ Not supported	☒ Supported by bundlers
Browser Support	☒ No (needs bundler)	☒ Yes (modern browsers)
this in Module	References <code>module.exports</code>	undefined
__dirname, __filename	☒ Available	

		✗ Not available (use import.meta.url)
Default in Node.js	☒ Yes (traditional)	⚠ Opt-in with configuration

Example 11: Same Module in Both Formats

```
// user-manager-cjs.js (CommonJS)
const users = [];

function addUser(name) {
  const user = { id: users.length + 1, name };
  users.push(user);
  return user;
}

function getUsers() {
  return users;
}

module.exports = { addUser, getUsers };
```

```
// user-manager-esm.js (ES Module)
const users = [];

export function addUser(name) {
  const user = { id: users.length + 1, name };
  users.push(user);
  return user;
}

export function getUsers() {
  return users;
}
```

```
// Using CommonJS version
const { addUser, getUsers } = require('./user-manager-cjs.js');
```



```
addUser('Alice');
addUser('Bob');
console.log('CJS Users:', getUsers());
```

```
// Using ES Module version (in a .mjs file or with "type": "module")
import { addUser, getUsers } from './user-manager-esm.js';

addUser('Alice');
addUser('Bob');
console.log('ESM Users:', getUsers());
```

Output:

```
CJS Users: [ { id: 1, name: 'Alice' }, { id: 2, name: 'Bob' } ]
ESM Users: [ { id: 1, name: 'Alice' }, { id: 2, name: 'Bob' } ]
```

4. Dynamic Imports: The Bridge Between Systems

What are Dynamic Imports?

Dynamic imports allow you to load modules on-demand at runtime, rather than at the beginning of your program. They work in both CommonJS and ES modules, returning a Promise that resolves to the module.

Example 12: Basic Dynamic Import

```
// heavy-module.js
export function processLargeData(data) {
  console.log('Processing large dataset...');
  // Simulate heavy processing
  const result = data.map(item => item * 2);
  return result;
}

export function analyzeData(data) {
  console.log('Analyzing data...');
```

```
    return {
      sum: data.reduce((a, b) => a + b, 0),
      average: data.reduce((a, b) => a + b, 0) / data.length
    };
  }

  console.log('Heavy module loaded!');
```

```
// app-with-dynamic-import.js
console.log('App started');

async function handleUserRequest(needsHeavyProcessing) {
  console.log('Handling request...');

  if (needsHeavyProcessing) {
    console.log('Loading heavy module...');

    // Load module only when needed
    const heavyModule = await import('./heavy-module.js');

    const data = [1, 2, 3, 4, 5];
    const processed = heavyModule.processLargeData(data);
    const analysis = heavyModule.analyzeData(processed);

    console.log('Processed:', processed);
    console.log('Analysis:', analysis);
  } else {
    console.log('Simple request, no heavy processing needed');
  }
}

// Test with simple request first
console.log('\n--- Request 1: Simple ---');
handleUserRequest(false);

// Then with heavy processing
setTimeout(() => {
  console.log('\n--- Request 2: Heavy Processing ---');
  handleUserRequest(true);
}, 1000);
```

Output:

```
App started

--- Request 1: Simple ---
Handling request...
Simple request, no heavy processing needed

--- Request 2: Heavy Processing ---
Handling request...
Loading heavy module...
Heavy module loaded!
Processing large dataset...
Analyzing data...
Processed: [ 2, 4, 6, 8, 10 ]
Analysis: { sum: 30, average: 6 }
```

Notice: The heavy module is only loaded when needed, not at app startup. This improves initial load time.

Example 13: Conditional Module Loading

```
// dev-logger.js
export function log(message) {
  console.log(`[DEV] ${new Date().toISOString()} - ${message}`);
}

export function debug(data) {
  console.log('[DEBUG]', JSON.stringify(data, null, 2));
}
```

```
// prod-logger.js
export function log(message) {
  // In production, might send to external logging service
  console.log(`[PROD] ${message}`);
}

export function debug(data) {
  // In production, debug might be disabled
}
```

```
// app.js - Load different modules based on environment
const ENV = process.env.NODE_ENV || 'development';

async function getLogger() {
  if (ENV === 'production') {
    return await import('./prod-logger.js');
  } else {
    return await import('./dev-logger.js');
  }
}

async function main() {
  const logger = await getLogger();

  logger.log('Application started');
  logger.log(`Running in ${ENV} mode`);
  logger.debug({ user: 'Alice', action: 'login' });
}

main();
```

Output:

```
[DEV] 2026-02-17T10:30:00.000Z - Application started
[DEV] 2026-02-17T10:30:00.000Z - Running in development mode
[DEBUG] {
  "user": "Alice",
  "action": "login"
}
```

Example 14: Dynamic Imports in CommonJS

Dynamic imports work even in CommonJS modules!

```
// Using dynamic import in a CommonJS file
// main.cjs (CommonJS file)

console.log('CommonJS module starting...');
```

```
async function loadESModule() {
  try {
    // Import an ES module from CommonJS
    const esmModule = await import('./utils.js');

    console.log('Successfully loaded ES module from CommonJS!');
    console.log('Result:', esmModule.formatDate(new Date()));

  } catch (error) {
    console.error('Failed to load module:', error);
  }
}

loadESModule();

// You can still use require for CommonJS modules
const path = require('path');
console.log('Current directory:', path.basename(__dirname));
```

Output:

```
CommonJS module starting...
Current directory: my-project
Successfully loaded ES module from CommonJS!
Result: 2026-02-17
```

Key Advantage: Dynamic imports allow CommonJS code to import ES modules, which is not possible with regular `require()`.

Example 15: Plugin System with Dynamic Imports

```
// plugins/email-plugin.js
export default {
  name: 'email',
  version: '1.0.0',

  async send(to, subject, body) {
    console.log(`Sending email to ${to}`);
    console.log(`Subject: ${subject}`);
    console.log(`Body: ${body}`);
  }
}
```

```
    return { success: true, messageId: 'msg_123' };  
  }  
};
```

```
// plugins/sms-plugin.js  
export default {  
  name: 'sms',  
  version: '1.0.0',  
  
  async send(to, message) {  
    console.log(`Sending SMS to ${to}`);  
    console.log(`Message: ${message}`);  
    return { success: true, messageId: 'sms_456' };  
  }  
};
```

```
// plugin-manager.js  
class PluginManager {  
  constructor() {  
    this.plugins = new Map();  
  }  
  
  async loadPlugin(pluginName) {  
    if (this.plugins.has(pluginName)) {  
      console.log(`Plugin '${pluginName}' already loaded`);  
      return this.plugins.get(pluginName);  
    }  
  
    console.log(`Loading plugin: ${pluginName}...`);  
  
    try {  
      const module = await import(`./plugins/${pluginName}-plugin.js`);  
      const plugin = module.default;  
  
      this.plugins.set(pluginName, plugin);  
      console.log(`Plugin '${plugin.name}' v${plugin.version} loaded successfully`);  
  
      return plugin;  
    } catch (error) {  
      console.error(`Failed to load plugin '${pluginName}':`, error.message);  
      return null;  
    }  
  }  
}
```

```
getPlugin(pluginName) {
  return this.plugins.get(pluginName);
}

// Usage
async function demo() {
  const manager = new PluginManager();

  // Load plugins on-demand
  const emailPlugin = await manager.loadPlugin('email');
  if (emailPlugin) {
    await emailPlugin.send('user@example.com', 'Hello', 'Welcome!');
  }

  console.log('\n--- Loading SMS Plugin ---');
  const smsPlugin = await manager.loadPlugin('sms');
  if (smsPlugin) {
    await smsPlugin.send('+1234567890', 'Your code is 123456');
  }

  console.log('\n--- Trying to load email again ---');
  await manager.loadPlugin('email'); // Should use cached version
}

demo();
```

Output:

```
Loading plugin: email...
Plugin 'email' v1.0.0 loaded successfully
Sending email to user@example.com
Subject: Hello
Body: Welcome!

--- Loading SMS Plugin ---
Loading plugin: sms...
Plugin 'sms' v1.0.0 loaded successfully
Sending SMS to +1234567890
Message: Your code is 123456
```

```
--- Trying to load email again ---  
Plugin 'email' already loaded
```

Use Cases for Dynamic Imports:

- Loading heavy modules only when needed (code splitting)
- Conditional module loading based on environment or feature flags
- Plugin systems where plugins are loaded on-demand
- Lazy loading in web applications to improve performance
- Loading ES modules from CommonJS code

5. Hybrid Usage: Working with Both Systems

Interoperability Between CJS and ESM

In real projects, you often need to work with both module systems. Understanding how they interact is crucial.

Example 16: Importing CommonJS from ES Modules

```
// cjs-module.js (CommonJS)  
const config = {  
  apiUrl: 'https://api.example.com',  
  timeout: 5000  
};  
  
function getConfig() {  
  return config;  
}  
  
module.exports = { config, getConfig };
```

```
// esm-consumer.js (ES Module with "type": "module")  
// Importing CommonJS module from ES module  
import cjsModule from './cjs-module.js';
```



```
console.log('Imported CJS module:', cjsModule);
console.log('Config:', cjsModule.config);
console.log('Get Config:', cjsModule.getConfig());

// Named imports work too
import { config, getConfig } from './cjs-module.js';
console.log('Direct config access:', config);
```

Output:

```
Imported CJS module: { config: { apiUrl: 'https://api.example.com', timeout: 5000 }, getConfig: [Function:
getConfig] }
Config: { apiUrl: 'https://api.example.com', timeout: 5000 }
Get Config: { apiUrl: 'https://api.example.com', timeout: 5000 }
Direct config access: { apiUrl: 'https://api.example.com', timeout: 5000 }
```

Rule: ES modules can import CommonJS modules, and both default and named imports work.

Example 17: Importing ES Modules from CommonJS (Using Dynamic Import)

```
// esm-module.js (ES Module)
export const version = '2.0.0';

export function greet(name) {
  return `Hello, ${name}!`;
}

export default {
  appName: 'MyApp',
  version
};
```

```
// cjs-consumer.js (CommonJS)
// Cannot use static require for ES modules
// const esmModule = require('./esm-module.js'); // ✗ This will error!

// Must use dynamic import
```

```
async function loadESM() {
  console.log('Loading ES module from CommonJS...');

  const esmModule = await import('./esm-module.js');

  console.log('Default export:', esmModule.default);
  console.log('Named export - version:', esmModule.version);
  console.log('Named export - greet:', esmModule.greet('Alice'));
}

loadESM();
```

Output:

```
Loading ES module from CommonJS...
Default export: { appName: 'MyApp', version: '2.0.0' }
Named export - version: 2.0.0
Named export - greet: Hello, Alice!
```

Rule: CommonJS cannot use `require()` for ES modules. Must use dynamic `import()` which returns a Promise.

Example 18: Creating Dual-Compatible Modules

Sometimes you need to create a module that works in both systems. Here's a pattern:

```
// dual-module.mjs (ES Module version)
export function multiply(a, b) {
  return a * b;
}

export function divide(a, b) {
  if (b === 0) throw new Error('Division by zero');
  return a / b;
}

export default { multiply, divide };
```

```
// dual-module.cjs (CommonJS version)
function multiply(a, b) {
  return a * b;
}

function divide(a, b) {
  if (b === 0) throw new Error('Division by zero');
  return a / b;
}

module.exports = { multiply, divide };
```

```
// package.json
{
  "name": "dual-module-package",
  "version": "1.0.0",
  "main": "./dual-module.cjs",
  "exports": {
    ".": {
      "import": "./dual-module.mjs",
      "require": "./dual-module.cjs"
    }
  }
}
```

How it Works:

- The `exports` field tells Node.js which file to use for each system
- When imported as ES module → uses `dual-module.mjs`
- When required as CommonJS → uses `dual-module.cjs`
- Provides maximum compatibility

Example 19: Migrating from CommonJS to ES Modules

Here's a practical example of migrating a project step by step.

```
// Step 1: Original CommonJS code
// old-app.js
const express = require('express');
```

```
const userController = require('./controllers/user');
const config = require('./config');

const app = express();

app.get('/users', userController.getAll);
app.listen(config.port);
```

```
// Step 2: Add "type": "module" to package.json
{
  "type": "module"
}
```

```
// Step 3: Convert to ES modules
// new-app.js
import express from 'express';
import * as userController from './controllers/user.js'; // Note: add .js
import config from './config.js';

const app = express();

app.get('/users', userController.getAll);
app.listen(config.port);
```

```
// Step 4: Handle __dirname (not available in ES modules)
// old way (CommonJS)
const path = require('path');
const configPath = path.join(__dirname, 'config.json');

// new way (ES modules)
import { fileURLToPath } from 'url';
import { dirname, join } from 'path';

const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
const configPath = join(__dirname, 'config.json');
```

Best Practices

1. Choose the Right Module System

```
// ☒ Use ES Modules for new projects
// Modern, standardized, better tooling support
import { myFunction } from './module.js';

// ☒ Use CommonJS when:
// - Working with older Node.js versions (< 12)
// - Many dependencies are CommonJS-only
// - Team is not ready to migrate
const myModule = require('./module');
```

2. Be Consistent Within a Project

```
// ☒ Bad: Mixing systems unnecessarily
// file1.js (ES Module)
export const foo = 'bar';

// file2.js (CommonJS)
module.exports = { baz: 'qux' };

// ☒ Good: Pick one system
// All files use ES Modules
export const foo = 'bar';
export const baz = 'qux';
```

3. Always Add File Extensions for ES Modules

```
// ☒ Bad: Missing extension
import { helper } from './utils';

// ☒ Good: Include extension
import { helper } from './utils.js';
```

4. Use Dynamic Imports for Code Splitting

```
// ☒ Good: Load heavy modules only when needed
async function handleAdminAction() {
  const adminTools = await import('./admin-tools.js');
  return adminTools.performAction();
}

// ☒ Bad: Loading everything upfront
import * as adminTools from './admin-tools.js'; // Loaded even if never used
```

5. Handle Errors in Dynamic Imports

```
// ☒ Good: Proper error handling
async function loadPlugin(name) {
  try {
    const plugin = await import(`./plugins/${name}.js`);
    return plugin;
  } catch (error) {
    console.error(`Failed to load plugin: ${name}`, error);
    return null;
  }
}

// ☒ Bad: No error handling
async function loadPlugin(name) {
  const plugin = await import(`./plugins/${name}.js`); // May crash
  return plugin;
}
```

Common Pitfalls and Solutions

Pitfall 1: Circular Dependencies

```
// ☒ Problem: Circular dependency
// a.js
const b = require('./b');
exports.valueA = 'A';
```

```
console.log('In A, b.valueB =', b.valueB); // undefined!

// b.js
const a = require('./a');
exports.valueB = 'B';
console.log('In B, a.valueA =', a.valueA); // works

// ☒ Solution: Restructure or use dynamic imports
// a.js
exports.valueA = 'A';

async function useB() {
  const b = await import('./b.js');
  console.log('In A, b.valueB =', b.valueB); // works!
}
```

Pitfall 2: Modifying Imported Values

```
// config.js
export let timeout = 5000;

// app.js
import { timeout } from './config.js';

// ☒ This won't work in ES modules
timeout = 10000; // Error: Assignment to constant variable

// ☒ Solution: Export a function to modify
// config.js
let timeout = 5000;
export function setTimeout(value) {
  timeout = value;
}
export function getTimeout() {
  return timeout;
}
```

Pitfall 3: Forgetting await with Dynamic Imports

```
// ☒ Bad: Forgetting await
async function loadModule() {
  const module = import('./module.js'); // Returns Promise!
```

```
module.doSomething(); // Error: module.doSomething is not a function
}

// ☒ Good: Using await
async function loadModule() {
  const module = await import('./module.js');
  module.doSomething(); // Works!
}
```

Interview Questions

Theoretical Questions

1. What is the difference between `module.exports` and `exports` in CommonJS?

Answer: `module.exports` is the actual object that gets returned when a module is required. `exports` is a reference to `module.exports`. You can add properties to `exports` (e.g., `exports.foo = 'bar'`), but if you reassign `exports` (e.g., `exports = { foo: 'bar' }`), it breaks the reference and won't work. For exporting a single value or completely replacing the `exports`, you must use `module.exports`. Rule: use `exports` for adding properties, `module.exports` for replacing the entire export.

2. How does Node.js resolve module paths?

Answer: Node.js follows this resolution order: (1) Core modules (like 'fs', 'path') are checked first, (2) If path starts with './', '../', or '/', it's treated as a file path, (3) Otherwise, Node.js looks in `node_modules` folders, starting from current directory and moving up, (4) For files, Node.js tries exact match, then adds `.js`, `.json`, and `.node` extensions, (5) For folders, it looks for `package.json` with `main` field, or `index.js`. This allows Node.js to find modules without specifying full paths.

3. What happens when a module is required multiple times?

Answer: Modules are cached after the first require. When a module is required the first time, Node.js executes the code and caches the result in `require.cache`. Subsequent requires return the cached module without re-executing the code. This means: (1) Module code runs only once, (2) All requires get the same module instance, (3) State is shared across all requires, (4) Performance is improved by avoiding redundant execution. This is why multiple requires of the same module share state.

4. What are the advantages of ES Modules over CommonJS?

Answer: ES Modules offer several advantages: (1) Static analysis - imports are analyzed at compile-time, enabling better tooling and tree-shaking to remove unused code, (2) Standardized - official JavaScript standard supported by browsers and Node.js, (3) Async loading - designed for async loading suitable for web, (4) Read-only bindings - imported values can't be accidentally modified, (5) Better IDE support - static imports enable better autocomplete and refactoring, (6) Named exports - more explicit than CommonJS, (7) Future-proof - the direction JavaScript is moving.

5. How do you enable ES Modules in Node.js?

Answer: There are three ways: (1) Add "type": "module" to package.json - makes all .js files ES modules, (2) Use .mjs file extension - individual files are treated as ES modules, (3) Use --input-type=module flag when running Node. Once enabled, you must use import/export instead of require/module.exports, include file extensions in imports, and adapt to differences like no __dirname. The recommended approach for new projects is "type": "module" in package.json.

6. Can you use require() in ES modules? Can you use import in CommonJS?

Answer: No and conditionally yes. In ES modules, require() is not available - you must use import statements. However, you can use dynamic import() which works everywhere. In CommonJS, you cannot use static import statements, but you CAN use dynamic import() which returns a Promise. This makes dynamic import() the universal solution that works in both systems. Example: const mod = await import('./module.js') works in both CJS and ESM.

7. What is tree-shaking and why does it only work with ES Modules?

Answer: Tree-shaking is the process of removing unused code from the final bundle. It only works with ES Modules because: (1) ES Module imports are static - can be analyzed at compile-time to determine which exports are actually used, (2) CommonJS requires are dynamic - can happen at runtime with computed paths, making it impossible to analyze statically, (3) ES Modules have explicit named exports - bundlers can track exactly what's imported, (4) CommonJS uses object assignment - entire module is included even if only one function is used. Tree-shaking can reduce bundle size by 50% or more.

8. What are dynamic imports and when should you use them?

Answer: Dynamic imports use import() as a function (not statement) that returns a Promise resolving to the module. Use cases: (1) Code splitting - load heavy modules only when needed to improve initial load time, (2) Conditional loading - load different modules based on runtime conditions, (3) Loading ES modules from CommonJS - only way to import ESM in CJS, (4) Plugin systems - load plugins on-demand, (5) Route-based loading - load components when navigating to routes. Dynamic imports enable lazy loading and reduce initial bundle size, improving performance.

9. What is the purpose of the "exports" field in package.json?

Answer: The exports field provides fine-grained control over what can be imported from a package and enables conditional exports. It allows: (1) Dual package support - serve different files for ESM vs CJS (e.g., "import": "./index.mjs", "require": "./index.cjs"), (2) Subpath exports - control exactly which files can be imported, (3) Environment-specific code - different code for Node.js vs browsers, (4) Encapsulation - hide internal modules by not exporting them. This replaced the older "main" field pattern and is the modern standard for package exports.

10. How do you handle __dirname and __filename in ES Modules?

Answer: In ES Modules, __dirname and __filename are not available. Instead, use import.meta.url which contains the file URL. Convert to path: import { fileURLToPath } from 'url'; import { dirname } from 'path'; const __filename = fileURLToPath(import.meta.url); const __dirname = dirname(__filename); This gives you the equivalent values. import.meta is a special object available in ES modules containing metadata about the current module, with url being the module's URL.

Practical Questions for Students to Solve

1. Question 1: Create a Module System

Create a simple user management system with separate modules:

- user.js - User class with properties (id, name, email)
- user-repository.js - Functions to add, remove, and find users (uses an array)
- validator.js - Email validation function
- app.js - Main file that uses all modules

Implement this twice: once using CommonJS and once using ES Modules.

2. Question 2: Module Caching Investigation

Create a module called counter.js that:

- Has a private counter variable starting at 0
- Exports increment(), decrement(), and getValue() functions
- Logs "Module initialized" when first loaded

Then create two files that both require/import this module. Demonstrate that they share the same counter state and that "Module initialized" only appears once.

3. Question 3: Build a Plugin Loader

Create a plugin system using dynamic imports:

- A PluginManager class that can load plugins by name
- Two sample plugins (e.g., logger-plugin.js, database-plugin.js)
- Each plugin should export a default object with name, version, and init() function
- The manager should cache loaded plugins and handle loading errors gracefully
- Demonstrate loading plugins on-demand

4. Question 4: Convert CommonJS to ES Modules

Given this CommonJS code, convert it to ES Modules:

```
const fs = require('fs');
const path = require('path');
const utils = require('./utils');

function readConfig() {
  const configPath = path.join(__dirname, 'config.json');
  const data = fs.readFileSync(configPath, 'utf8');
  return JSON.parse(data);
}

module.exports = { readConfig };
```

Include proper handling of __dirname replacement.

5. Question 5: Implement Conditional Module Loading

Create a configuration loader that:

- Loads different config modules based on NODE_ENV (development, production, test)
- Uses dynamic imports to load the appropriate config
- Provides fallback to default config if environment-specific config doesn't exist
- Create sample config files: config-dev.js, config-prod.js, config-default.js
- Each config should export different values (API URLs, timeouts, etc.)

Summary

The module system is fundamental to organizing and structuring Node.js applications. Understanding both CommonJS and ES Modules, along with their interoperability, is essential for modern JavaScript development.

Key Takeaways:

- **CommonJS** uses `require/module.exports`, is synchronous, and has been Node.js's default since the beginning
- **ES Modules** use `import/export`, are asynchronous, and represent the JavaScript standard
- Modules are cached after first load, improving performance and enabling state sharing
- ES Modules require file extensions in imports (`.js`, `.mjs`)
- ES Modules enable tree-shaking for smaller bundle sizes
- Dynamic imports work in both systems and enable code splitting
- ES Modules can import CommonJS, but CommonJS needs dynamic import for ES Modules
- Enable ES Modules with `"type": "module"` in `package.json` or `.mjs` extension
- `__dirname` and `__filename` are replaced with `import.meta.url` in ES Modules
- Use `exports` field in `package.json` for dual CJS/ESM support
- Choose ES Modules for new projects, CommonJS for legacy compatibility
- Dynamic imports enable conditional loading, lazy loading, and plugin systems 

Mastering the module system allows you to write maintainable, scalable applications with proper code organization, reusability, and performance optimization through code splitting and tree-shaking.